GIT-DOCS · 11

# Testing Rollback

A tag pointed at an older commit, pushed once — and the running stack genuinely went backward, with no rebuild and no manual intervention.

tag: v0.1.0-rollback → commit 811af77  2026-08-16

## 00 What rollback actually is

Not a feature. Not a separate code path. `deploy.yml` has exactly one job, and it has no idea whether a given tag is a "release" or a "rollback" — that distinction exists only in the human's head, in which commit they chose to point the tag at. A rollback tag pointed backward runs through the identical resolve → pull → up sequence as any other deploy. The only reason this works without rebuilding anything is doc 3's design choice from the very start of the day: every push builds and tags by commit SHA, whether or not that commit ever becomes a release — so an old SHA's images are usually still sitting in the registry, untouched, whenever you need them again.

## 01 What was actually tested

Currently deployed: `v0.1.0`, resolving to commit `9992ba4`. Rollback target chosen deliberately: `811af77` — doc 9's confirmed fully-successful build, where all five images actually finished pushing (unlike `60bac55`, where only one image was ever manually pushed as a diagnostic, not the full set). Real rollback needs a real, complete prior version to land on.

```
git tag v0.1.0-rollback 811af77
git push origin v0.1.0-rollback
```

## 02 Which components are involved

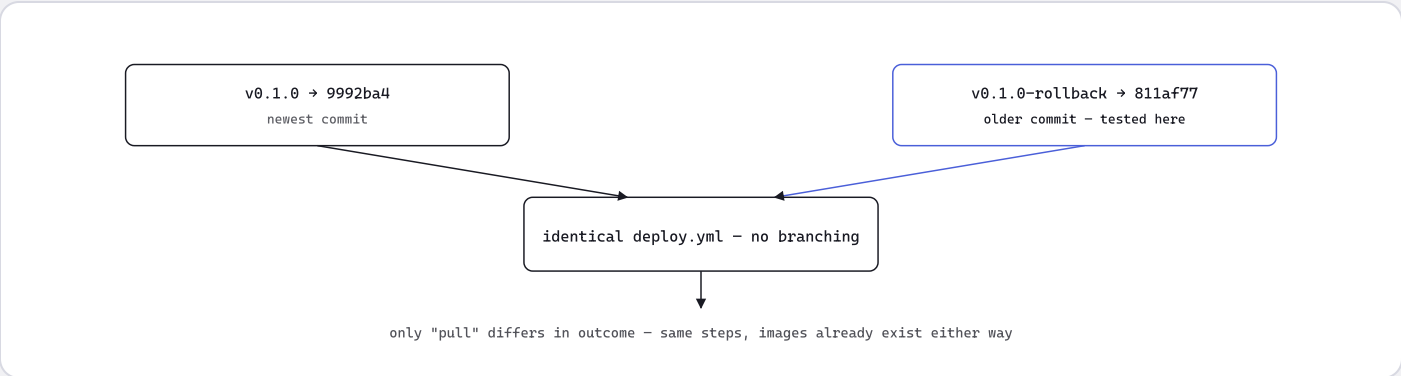
| COMPONENT | ROLE IN A ROLLBACK |
|---|---|
| GitHub (router + controller) | Matches the tag push, resolves `deploy.yml` into a job — no awareness this is "a rollback" |
| Runner, on clubuntu | Picks up the job, resolves the tag to a commit SHA, sets `IMAGE_TAG` |
| Registry | Already holds the old SHA's images from when they were originally built — only ever *read* here, never written to |
| Docker Compose | Diffs the new image references against what's running; recreates only the containers whose image actually changed |
| Build pipeline | Not involved at all — this is the whole point |

## 03 What actually changed, container by container

| CONTAINER | BEFORE | AFTER | RECREATED? |
|---|---|---|---|
| generic-mcp-gateway | `gateway:9992ba4` | `gateway:811af77` | **Yes** |
| coder-commands-mcp | `… :9992ba4` | `… :811af77` | **Yes** |
| codercommands-docker-mcp | `docker-mcp:9992ba4` | `docker-mcp:811af77` | **Yes** |
| codercommands-postgres-mcp | `… :9992ba4` | `… :811af77` | **Yes** |
| codercommands-sync | `sync:9992ba4` | `sync:811af77` | **Yes** |
| codercommands-postgres | `postgres:17` | `postgres:17` | No — image never changed |

`postgres` staying untouched (still showing its original uptime, not recreated) is Compose's own diffing at work, not a special case written for this project — it only restarts what actually needs restarting.

## 04 The mechanism



Same picture as doc 4's original plan, and the FAQ's answer to "what happens during a rollback tag" — now backed by two real, confirmed deploys instead of description alone.

## 05 Verified

```
$ curl http://clubuntu.dala-cirius.ts.net:8100/healthz
{"status":"ok"}
```

Reached directly over Tailscale from the dev laptop this time, not just from inside clubuntu — confirming the gateway is genuinely reachable across the tailnet on its published port, not only reachable locally.

## 06 What this proves

- Rollback cost one pull of already-built images — no rebuild, no wait for a fresh `docker buildx bake`
- Compose only recreated the five containers whose image reference actually changed, left `postgres` running untouched
- The exact same `deploy.yml`, unmodified, correctly served both a forward release and a backward rollback — because it was never told the difference

## 07 Cleanup exposed a gap — tags aren't state

`v0.1.0-rollback` was only ever meant as a test, so it was deleted once confirmed: `git tag -d` locally, `git push origin --delete` remotely. Deleting a tag fires no workflow — only pushing one does — so clubuntu kept running exactly what it was running before the delete: `811af77`. But the only tag left, `v0.1.0`, pointed at `9992ba4`. The label and the deployed reality had quietly drifted apart, and nothing about deleting a tag would ever fix that on its own.


A tag is a pointer, nothing more — deleting one is a git-history edit, not an instruction to any running system.

## 08 Re-pushing an unchanged tag does nothing — why, and the fix

The instinct is `git push origin v0.1.0` again. That does nothing: the ref on GitHub already points at `9992ba4`, so there's no actual change to push — Git reports "Everything up-to-date," and no push event fires for GitHub to dispatch a job from. Resyncing needed a genuinely new tag-push event: delete `v0.1.0`, recreate it pointing at the exact same commit, push it fresh.

```
git tag -d v0.1.0
git push origin --delete v0.1.0
git tag v0.1.0 9992ba4
git push origin v0.1.0 # a genuinely new ref creation — this
one does dispatch
```

| CONTAINER | BEFORE REDEPLOY | AFTER REDEPLOY |
|---|---|---|
| generic-mcp-gateway | `gateway:811af77` | `gateway:9992ba4` |
| coder-commands-mcp | `… :811af77` | `… :9992ba4` |
| codercommands-docker-mcp | `docker-mcp:811af77` | `docker-mcp:9992ba4` |
| codercommands-postgres-mcp | `… :811af77` | `… :9992ba4` |
| codercommands-sync | `sync:811af77` | `sync:9992ba4` |
| codercommands-postgres | unaffected both times — 11+ minutes uptime throughout | |

```
$ curl http://localhost:8100/healthz
{"status":"ok"}
```

Label and reality back in agreement — `v0.1.0` says `9992ba4`, clubuntu runs `9992ba4`.

```
git-docs — 11 · tag: v0.1.0-rollback → 811af77 · resynced to v0.1.0 → 9992ba4
```